ALPACA AI TRADING AGENTS HACKATHON · OPTIONS ALPHA AGENTS

# Options Alpha Agent

An LLM chooses the trades. A deterministic gate layer sits between its tool call and the broker and can veto any of them.

**Contribution:** the gate layer, not the strategy

**Stack:** Claude Opus 5 · Alpaca MCP server · alpaca-py · Streamlit

**Universe:** SPY / QQQ / IWM monthly defined-risk verticals

THE PROBLEM

# An agent that can trade is an agent that can misfire

Alpaca's MCP server exposes **72** tools. `place_option_order`, `close_all_positions` and `cancel_all_orders` sit in the same list as the quote endpoints.

— Hand the model that list and every risk rule you write becomes **decoration**, because there is a write path that never touches it.

— Prompt instructions are not a control. They are a request the model may decline, misread, or reason around.

— The interesting failure is not a bad opinion about SPY. It is a **correctly reasoned order that is twice the size the book allows**.

So the authority to place an order has to live somewhere the model cannot reach.

ARCHITECTURE

# One write path, and the model is not on it

| READS | JUDGEMENT | AUTHORITY | WRITES |
|---|---|---|---|
| **Alpaca MCP** | **Claude Opus 5** | **gates.py** | **broker.py** |
| 19 read-only tools, allowlisted from 72. Chain, greeks, IV, quotes, account. | Bounded session. Proposes a defined-risk vertical, or declines and says why. | Pure functions. No network, no account, no model. The veto is final. | The only file that can write to Alpaca. Checkable by grep. |

— **Allowlist, not denylist.** A denylist is correct until Alpaca ships a tool nobody has heard of, and it fails open that day.

— **Pure gates.** Everything arrives as an argument, so the safety-critical logic is the easiest thing in the repo to test.

— **Every decision, verdict and fill** is appended to an audit log, which the public dashboard replays.

THE GATE LAYER

# Sixteen gates, in two jobs

| | | |
|---|---|---|
| Paper only | Key prefix `PK`, client in paper mode, and `ALPACA_PAPER_TRADE=true`. All three. | BACKSTOP |
| Account role | No default. Unset is an error; the competition account was unreachable before kickoff. | BACKSTOP |
| Structure & expiry | Two legs, one underlying, opposing sides. At least 10 days out *and* the third Friday. | SHAPER |
| Quote sanity | Net price inside the spread width. A crossed or stale quote is rejected before any risk number is trusted. | BACKSTOP |
| Loss caps | Worst case **$1,500** per position, **$15,000** across the book, priced off the worst side of the quote. | BACKSTOP |
| Short delta | `abs(short_delta) ≤ 0.50`. Refuses to sell at or inside the money. | SHAPER |
| Book net delta | **$50,000** of underlying, priced off the quote *the broker reads*, not one the model supplies. | BACKSTOP |
| Kill switch & dedupe | A sentinel file checked before every write. No identical spread inside 30 minutes. | BACKSTOP |

EVIDENCE · THE VETO THAT FIRED

# The model sized to a rule it invented

**$3,496**
proposed: 4 lots, 10-wide QQQ call spread,
sized to a 3.5%-of-equity heuristic of its own

**$1,500**
the book's per-position cap,
which the model did not apply

**$1,308**
restructured: 3 lots, 5-wide,
after the veto reason went back to it

> *the gate was right and I was wrong*
>
> — the agent, in the audit log

The veto is not the interesting part. The interesting part is that a fluent, well-argued proposal was **more than twice the limit**, and nothing in the reasoning flagged it.

EVIDENCE · THE MODEL DOES NOT PRICE ITS OWN RISK

# One unchecked input fed the delta cap

`underlying_price` multiplies straight into book delta, and it arrived from the model with nothing checking it. A wrong ticker or a stale print moves the number the cap is measured against, silently, in either direction.

### What the agent did

Offered a QQQ last trade of `717.80`, it declined to pass it: the print was 40 shares pre-market, the quote sat at `712.34/712.44`, and the chain's greeks are struck off the quote. It passed `712.39`.

### What we changed

The broker now reads the underlying's quote midpoint itself and gates on that. Independently computed: `712.39`. A supplied value that disagrees is corrected and recorded, not used.

Good judgement in one session is luck. The fix generalises it so no session has to repeat it.

EVIDENCE · THE BUG THAT MATTERED MOST

# The aggregate gates could not see the account

They summed a list of open positions that only ever held orders placed **in the current process**. Every session started believing the book was empty.

— Aggregate loss summed to `$0`. Book net delta summed to `$0`. Dedupe could not see yesterday's spread.

— With a one-order-per-session cap the aggregate loss gate was not weak, it was **unreachable**: one position cannot exceed $15,000 when each is capped at $1,500.

— `positions.py` now rebuilds the book from what the account holds: parses OCC symbols, pairs legs into verticals, prices each from its entry fills and live greeks.

— Verified with a **negative control**: an order that passes against an empty book, and is vetoed once the real $1,293 position is loaded.

A gate nobody has watched fail is not a gate. Every limit here has vetoed something real.

METHOD

# Limits are derived, not tuned

### Where the numbers come from

Aggregate loss follows from a stated **15% drawdown tolerance**. Per-position follows from wanting roughly **ten concurrent slots**. Neither was fitted to a return.

### Sensitivity, measured

At three contracts, moving the per-position cap **$1k → $2k → $3k** shifts chain admissibility **58% → 90% → 99%**. Smooth, no cliff.

- Measured against **6,375** real verticals, median worst-case loss per contract is **$282** against a $1,500 cap. **The backstop is silent in normal operation, which is what a backstop should be.**
- `calibrate_limits.py` reproduces this and references no P&L, so it cannot be overfitted to returns.
- Deliberately **no IV gate**. An IV floor is predictive gating on premia. IV reaches the model and the audit log; it never vetoes.

# What the paper engine actually does

- **It fills market option orders, not limits.** A limit priced at the ask, marketable by definition, sat unfilled for 30 seconds. So the agent submits market orders and we say so, rather than claiming fills we would not get.
- **Multi-leg `MLEG` orders do fill.** A SPY 766/771 vertical filled at $2.90 net debit. The whole design rested on this, so it was worth three throwaway orders to establish.
- **Closing a vertical takes the short leg first.** Long leg first leaves the short momentarily uncovered and Alpaca rejects it (`40310000`); level 3 cannot hold a naked short. An agent closing in iteration order throws 403s live.
- **A naive expiry query never reaches the monthly.** SPY dailies fill the 500-row page cap, so the gate checks third-Friday alignment, not distance.
- **Live greeks and IV are published** on the chain snapshot; historical IV is not. Live selection yes, backtest no.

WHAT IS NOT CLAIMED

# No edge, and the trade we made knowingly

— **No edge is claimed.** The strategy is a worked example that exercises the machinery. Seven days does not contain a validation cycle, and Alpaca's options history cannot support a backtest.

— **Supervised autonomy costs P&L.** Bounded sessions, a human present, nothing scheduled, nothing surviving the process. That is a thinner ledger than a scheduled agent would produce.

— A seven-day P&L sample is noise either way. Buying a marginally better number by weakening the safety story would be a bad trade.

### Live dashboard

Read-only audit-log replay. Holds no credentials and has no code path to a broker.

### Repository

Gates, tests, calibration and the full write-up. 105 tests, ruff clean.